

CHURN PREDICTION

CODE:-

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_validate,
StratifiedKFold
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix, classification_report
)

# =====
# 1. SETUP & SYNTHETIC DATA GENERATION
# =====

print("Generating synthetic dataset...")
np.random.seed(42)
n_samples = 1000

data = pd.DataFrame({
    'Age': np.random.randint(18, 70, size=n_samples),
    'Tenure': np.random.randint(1, 72, size=n_samples),
    'MonthlyCharges': np.random.uniform(20, 120, size=n_samples),
    'ContractType': np.random.choice(['Month-to-month', 'One year', 'Two
year'], size=n_samples, p=[0.5, 0.3, 0.2]),
```

```
    'TechSupport': np.random.choice(['Yes', 'No'], size=n_samples, p=[0.4,
0.6]),
    'Churn': np.random.choice([0, 1], size=n_samples, p=[0.75, 0.25]) #
Imbalanced target (25% Churn)
})
```

```
print(f"Dataset Shape: {data.shape}\n")
print(data.head())
print("-" * 50)
```

```
# =====
# 2. PREPROCESSING & TRAIN/TEST SPLIT
# =====
```

```
print("Splitting and preprocessing data...")
X = data.drop(columns=['Churn'])
y = data['Churn']
```

```
num_features = ['Age', 'Tenure', 'MonthlyCharges']
cat_features = ['ContractType', 'TechSupport']
```

```
# Stratified split to preserve 75/25 target ratio in both train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
# Build preprocessing pipelines
num_transformer = Pipeline(steps=[('scaler', StandardScaler())])
cat_transformer = Pipeline(steps=[('encoder',
OneHotEncoder(drop='first'))])
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', num_transformer, num_features),
        ('cat', cat_transformer, cat_features)
    ]
)
```

```

)

# =====
# 3. MODEL BUILDING & CROSS-VALIDATION
# =====
print("Running 5-Fold Stratified Cross-Validation...")
models = {
    'Logistic Regression': LogisticRegression(random_state=42,
max_iter=1000),
    'Random Forest': RandomForestClassifier(random_state=42,
n_estimators=100)
}

scoring_metrics = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']
cv_results = {}

for name, model in models.items():
    # Build complete pipeline (Preprocessing + Estimator) to prevent data
leakage
    clf_pipeline = Pipeline(steps=[('preprocessor', preprocessor),
('classifier', model)])

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_validate(clf_pipeline, X_train, y_train, cv=cv,
scoring=scoring_metrics)

    # Store average scores across the 5 folds
    cv_results[name] = {metric: scores[f'test_{metric}'].mean() for metric
in scoring_metrics}

df_cv = pd.DataFrame(cv_results).T
print("\n--- 5-Fold Cross-Validation Results (Train Set) ---")
print(df_cv.round(3))
print("-" * 50)

```

```

# =====
# 4. FINAL TEST EVALUATION & PLOTS
# =====
print("Evaluating final performance on held-out Test Set...")
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
roc_curves = {}

for name, model in models.items():
    clf_pipeline = Pipeline(steps=[('preprocessor', preprocessor),
    ('classifier', model)])
    clf_pipeline.fit(X_train, y_train)

    # Generate final predictions
    y_pred = clf_pipeline.predict(X_test)
    y_prob = clf_pipeline.predict_proba(X_test)[:, 1]

    print(f"\n[ {name} Final Test Classification Report]")
    print(classification_report(y_test, y_pred))

    # Extract ROC data
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    roc_curves[name] = (fpr, tpr, roc_auc_score(y_test, y_prob))

    # Subplot selection for Confusion Matrices
    ax_idx = 0 if name == 'Logistic Regression' else 1
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[ax_idx])
    axes[ax_idx].set_title(f'{name} Confusion Matrix')
    axes[ax_idx].set_xlabel('Predicted Label')
    axes[ax_idx].set_ylabel('Actual Label')

plt.tight_layout()
plt.show()

# Plot Receiver Operating Characteristic (ROC) Curves

```

```

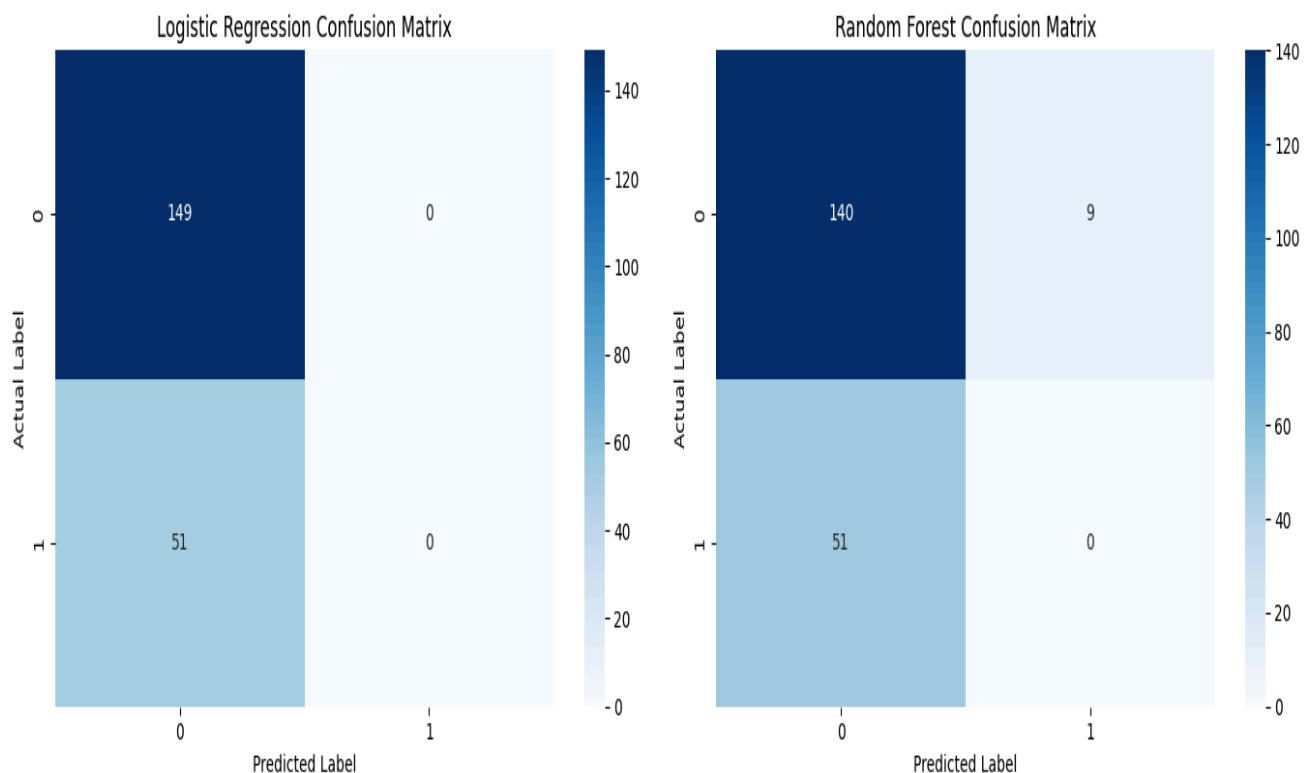
plt.figure(figsize=(8, 6))
for name, (fpr, tpr, auc_score) in roc_curves.items():
    plt.plot(fpr, tpr, label=f'{name} (AUC = {auc_score:.2f})')
plt.plot([0, 1], [0, 1], 'k--', label='Random Guess (AUC = 0.50)')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC Curve Comparison (Test Set)')
plt.legend()
plt.grid(True)
plt.show()

```

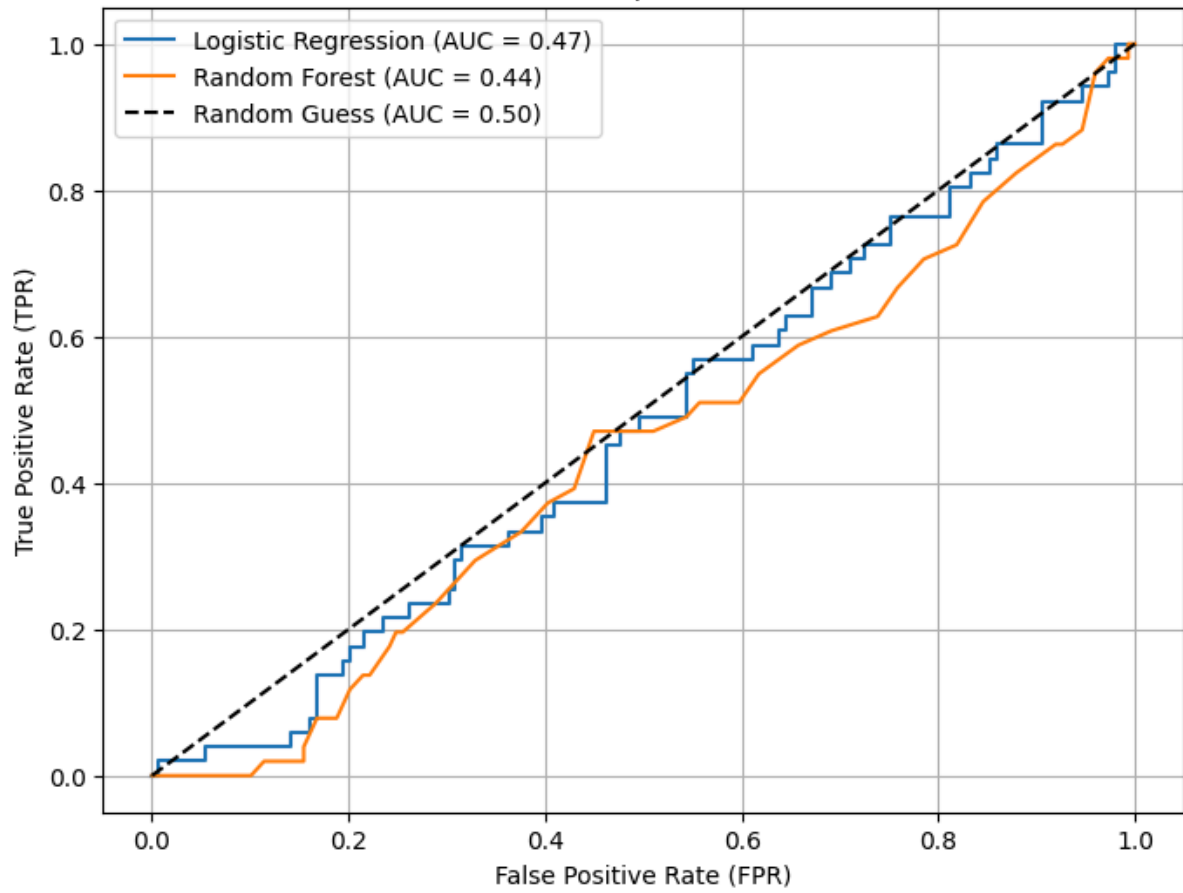
JUPYTER NOTEBOOK LINK:-

https://colab.research.google.com/drive/1RSvA4wAv_FWWxQNk4hXZCavcHZDmNrh_?usp=sharing

OUTPUT SCREENSHOTS:-



ROC Curve Comparison (Test Set)



EXPLANATION:-

Project Executive Summary: End-to-End Churn Prediction Pipeline

1. Project Overview

This project built a complete Machine Learning workflow divided into two distinct engineering phases:

Model Building & Evaluation: Training and evaluating a binary classifier to predict customer churn.

Operational Deployment: Packaging the predictive logic into a lightweight web API and containerizing it with Docker.

2. Model Development & Insights

The Pipeline: Handled an imbalanced dataset (25% churn) by creating an isolated processing pipeline. Numerical features were scaled using `StandardScaler` and categorical features encoded via `OneHotEncoder`.

Validation Strategy: Used an 80/20 train/test stratified split combined with 5-Fold Stratified Cross-Validation to prevent data leakage and evaluate performance.

Algorithm Comparison: Compared a linear baseline (Logistic Regression) against an ensemble method (Random Forest).

3. API Deployment & Containerization

To transition the model into a production environment, it was operationalized using modern MLOps tools:

FastAPI Microservice (app.py): Developed a lightweight, high-performance API. It enforces strict structural data checking via Pydantic schemas.

GET / — Microservice health check.

POST /predict — Accepts real-time customer data (Age, Tenure, Charges) and returns a structural churn probability.

Docker Containerization (Dockerfile): Built a deployment blueprint using a lightweight python:3.10-slim image to isolate dependencies and guarantee that the code runs identically on any cloud server.

}